A Practical Activity Report Submitted for
DATA ENGINEERING-(UCS677)

END-Semester Lab Evaluation

QuestBoard

A Freelance Micro-Task Marketplace for College Students

Submitted To

Mandeep Kaur

BE Third Year

Submitted by

Mygna Sai (102303127)

Batch: 3C15

Computer Science and Engineering Department

TIET, Patiala

January-May 2026

Contents

1. Introduction.....3
2. Tools and Technologies Used.....4
3. Workflow Diagram.....6
4. Database Schema.....7
5. Sample Screenshots.....8
6. Potential Improvements.....11
7. Conclusion.....12

1. Introduction

QuestBoard is a MongoDB-centric MERN project designed to act as a freelance micro-task marketplace built specifically for college students. The main objective of this project is to demonstrate how NoSQL databases, specifically MongoDB, can be used for efficient storage, retrieval, and analysis of marketplace data involving deeply nested arrays, multi-attribute filtering, and cross-collection joins.

On QuestBoard, students post small tasks such as design work, coding help, lecture notes, assignment review, or short writing jobs. Other students browse the open task board and place bids that include their proposed price, expected delivery time, and a short pitch. Each task document stores its bids as an embedded array, which allows MongoDB to handle the entire bidding lifecycle inside a single document while still supporting complex query and filter logic on individual bids.

The project focuses primarily on database engineering and analytics, while the frontend is intentionally kept minimal and acts only as a visualization layer for displaying task listings, bid details, and aggregation results.

Key Features

- Task posting with categories, skill tags, budget range, and deadline
- Embedded bid arrays stored directly inside each task document
- Logical and element operators to filter by skill, price, and deadline
- $lookup-based joins between tasks and bidder profiles
- CRUD operations for users, tasks, bids, and reviews
- Advanced MongoDB aggregation pipelines for marketplace analytics
- Indexing on skills, budget, and deadline fields for faster querying
- Replica set support for high availability
- Sharding support for horizontal scalability
- Minimal React dashboard for visualizing tasks, bids, and insights

This project demonstrates practical exposure to NoSQL schema design with embedded documents, multi-condition filtering, document joins, and database-side analytics using MongoDB.

2. Tools and Technologies Used

Frontend Framework

- React.js — Used to build a lightweight dashboard for displaying open tasks, bid lists, user profiles, and analytics outputs.

Backend Framework

- Node.js + Express.js — Used to build REST APIs, handle task posting and bidding logic, and connect application services with MongoDB.

Database

- MongoDB — Used as the primary NoSQL database for storing tasks, embedded bid arrays, user profiles, and review documents. MongoDB was chosen for its flexible schema, native support for arrays, and powerful aggregation framework.

ODM

- Mongoose — Used for schema definition, model creation, and structured interaction between Node.js and MongoDB collections.

HTTP Client

- Axios — Used in the frontend for consuming backend REST APIs.

API Testing

- Postman — Used for testing CRUD APIs, filter queries, and aggregation endpoints.

Database Visualization

- MongoDB Compass — Used to inspect collections, validate documents, and test aggregation pipelines visually.

Development Environment

- VS Code — Used for development and project management.

This technology stack ensures the project remains lightweight on the frontend while strongly emphasizing backend data engineering and MongoDB analytics.

3. Workflow Diagram

The following diagram illustrates the end-to-end data flow of the QuestBoard system, from user interactions on the React frontend to MongoDB collections, aggregation pipelines, and the final analytics views rendered on the dashboard.

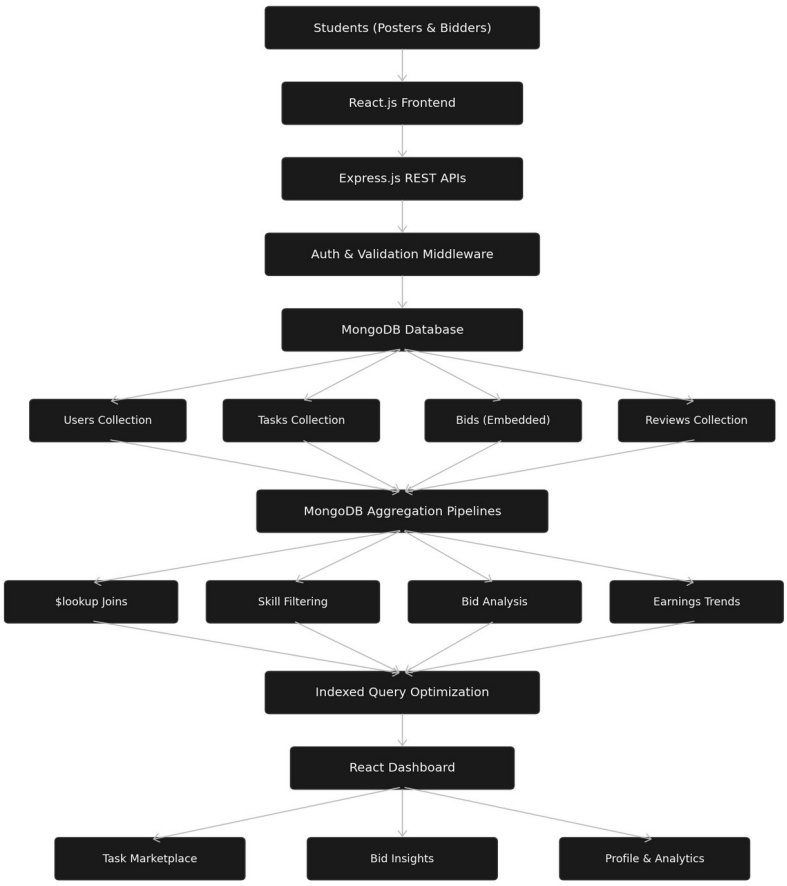


Figure 3.1 — End-to-end workflow of the QuestBoard system

4. Database Schema

The data model is organized around three primary collections — users, tasks, and reviews. Bids are stored as an embedded array inside each task document, which keeps related write operations atomic and avoids extra joins on the read path. Reference fields such as postedBy, bids.bidder, taskId, reviewer, and reviewee are resolved at query time using the $lookup aggregation stage.

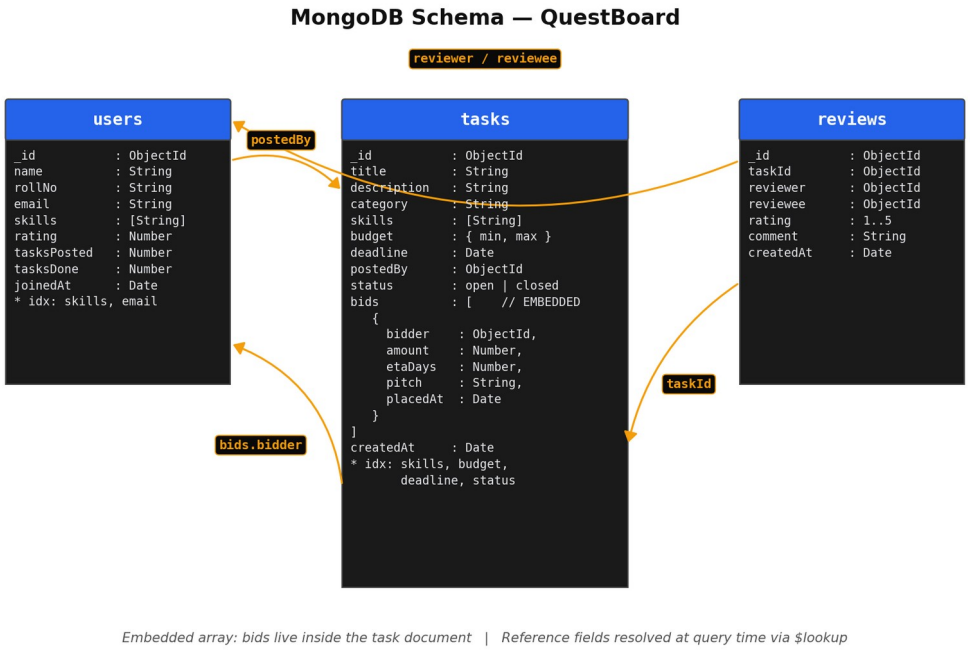


Figure 4.1 — MongoDB collection structure with embedded bids and reference relationships

Indexes are defined on high-cardinality and frequently filtered fields. The tasks collection is indexed on skills, budget, deadline, and status to support multi-condition queries with logical and element operators. The users collection is indexed on skills and email.

5. Sample Screenshots

This section illustrates representative views from the QuestBoard interface, demonstrating the marketplace listing, multi-operator filtering, $lookup-driven bidder profile joins, and aggregation outputs from the analytics page.

5.1 Task Marketplace

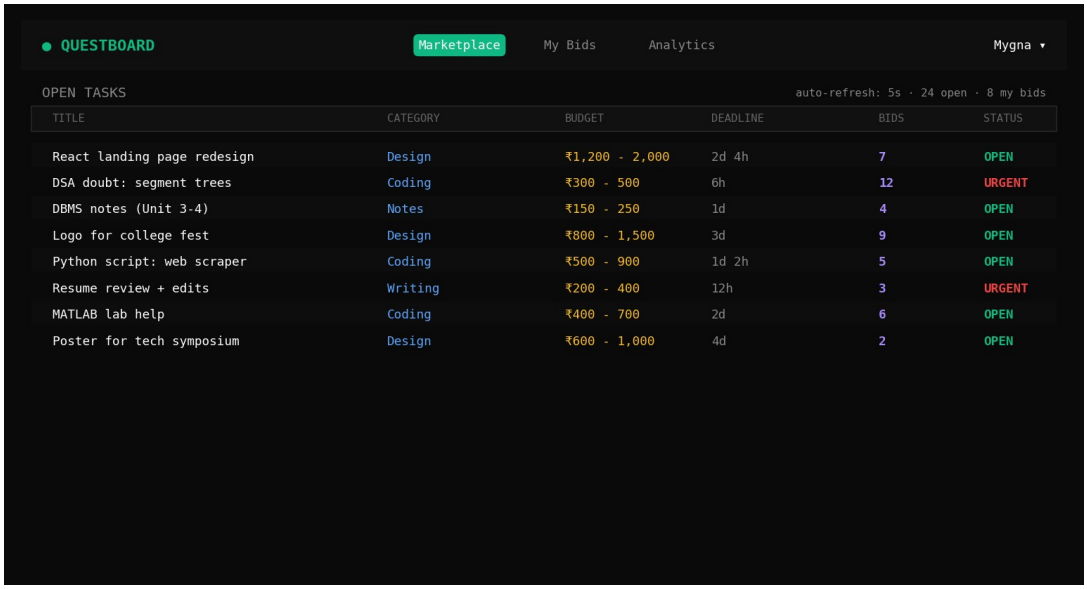
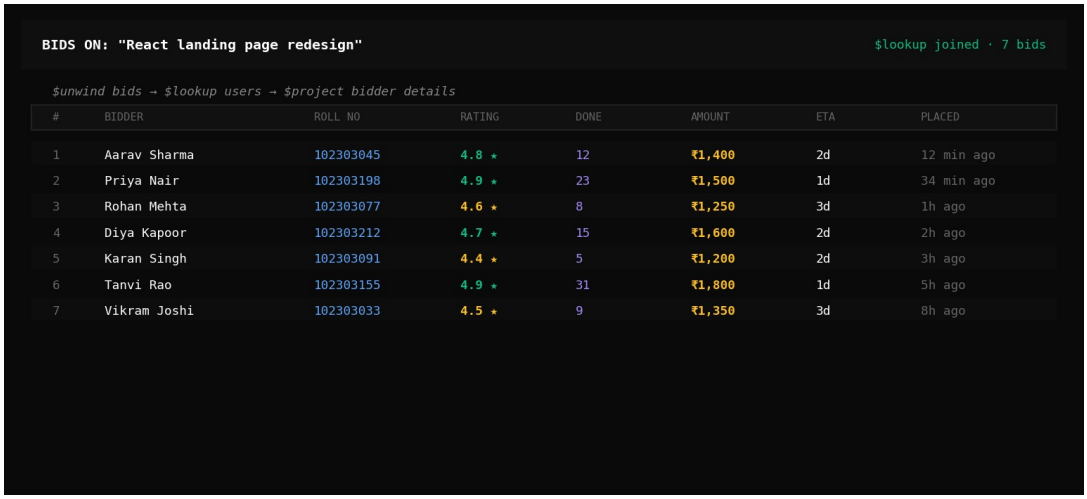


Figure 5.1 — Live list of open tasks with category, budget, deadline, and bid count

5.2 Bid Filtering with Logical & Element Operators

Figure 5.2 — UI selections translated into a MongoDB query using $and, $in, $gte, $lte, $gt

5.3 Bidder Profile View ($lookup Join)



#	BIDDER	ROLL NO	RATING	DONE	AMOUNT	ETA	PLACED
1	Aarav Sharma	102303045	4.8 ★	12	₹1,400	2d	12 min ago
2	Priya Nair	102303198	4.9 ★	23	₹1,500	1d	34 min ago
3	Rohan Mehta	102303077	4.6 ★	8	₹1,250	3d	1h ago
4	Diya Kapoor	102303212	4.7 ★	15	₹1,600	2d	2h ago
5	Karan Singh	102303091	4.4 ★	5	₹1,200	2d	3h ago
6	Tanvi Rao	102303155	4.9 ★	31	₹1,800	1d	5h ago
7	Vikram Joshi	102303033	4.5 ★	9	₹1,350	3d	8h ago

Figure 5.3 — Bids joined with user profiles via $unwind → $lookup → $project pipeline

5.4 Analytics — Average Winning Bid by Category

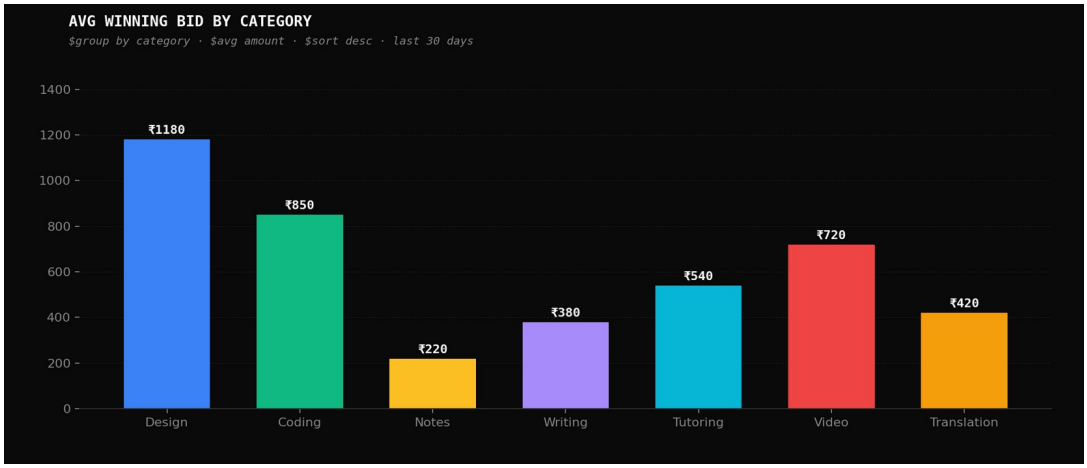


Figure 5.4 — Aggregation pipeline grouping by category over the last 30 days

5.5 Top Bidders Leaderboard

TOP BIDDERS — LEADERBOARD

$lookup users + reviews · combined score

joined tasks completed + avg rating + on-time delivery rate

RANK	NAME	SKILLS	DONE	RATING	ON-TIME	SCORE
#1	Tanvi Rao	React, Node, Mongo	31	4.9 ★	97%	98.2
#2	Priya Nair	Python, ML, Pandas	23	4.9 ★	94%	95.6
#3	Diya Kapoor	Figma, Illustrator	15	4.7 ★	96%	91.4
#4	Aarav Sharma	JS, React, Tailwind	12	4.8 ★	92%	89.8
#5	Rohan Mehta	C++, DSA, MATLAB	8	4.6 ★	100%	87.1

Figure 5.5 — Combined score from joined task completion, ratings, and on-time delivery

5.6 Weekly Marketplace Activity Trend

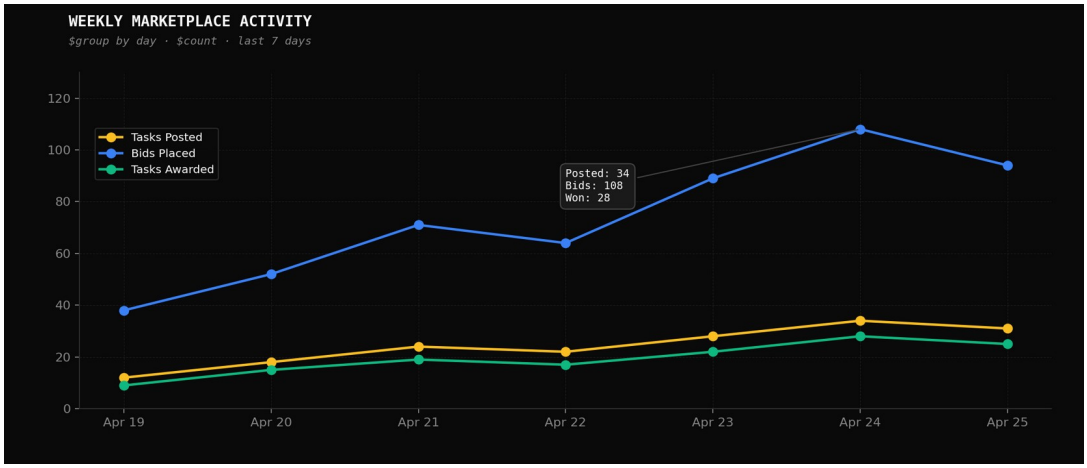


Figure 5.6 — Daily counts of tasks posted, bids placed, and tasks awarded over the past week

6. Potential Improvements

The frontend is intentionally minimal and focuses on displaying the processed outputs of the MongoDB backend.

Dashboard

Displays:

- Open tasks grouped by category
- Active bids placed by the current user
- Notifications for newly received bids and accepted offers

Analytics Page

Displays outputs of MongoDB aggregation pipelines such as:

- Most in-demand skill categories
- Average winning bid per task category
- Peak posting and bidding hours
- Top-rated bidders combined from joined user and review data

Future Enhancements

- Reputation scoring engine using weighted reviews and on-time delivery rate
- Real-time bid updates using WebSockets layered over MongoDB change streams
- Smart task recommendations based on a user's skill tags and past bids
- Escrow-style payment simulation between poster and selected bidder
- Full-text search on task titles and descriptions using MongoDB Atlas Search

API Output (Postman)

Postman is used to demonstrate:

- Task CRUD API responses
- Bid placement and acceptance API responses
- Filtered task queries using logical and element operators
- Aggregation results from analytics endpoints

These outputs validate the functionality of the backend and MongoDB operations.

7. Conclusion

QuestBoard demonstrates how MongoDB can be effectively used to build a real-world freelance marketplace tailored for college students. The project showcases practical NoSQL schema design with embedded bid arrays, multi-condition filtering using logical and element operators, cross-collection joins via $lookup, and analytics-style aggregation pipelines.

By focusing on MongoDB's core strengths such as flexible document storage, embedded arrays, expressive query operators, aggregation, replication, and sharding, the project highlights how NoSQL databases can support marketplace platforms where data is highly relational in spirit but naturally fits a document model.

The minimal frontend ensures that the main emphasis remains on backend architecture and database engineering, making this project a strong demonstration of practical NoSQL and data engineering concepts applied to a domain that students themselves can relate to.